

Projekt implementacji algorytmu CHAMELEON

Metody Eksploracji Danych — Etap 1

Oliwier Szypczyn

2026-04-26

Projekt implementacji algorytmu CHAMELEON

Autor: Oliwier Szypczyn

Przedmiot: Metody Eksploracji Danych, sem. letni 2025/2026, Politechnika Warszawska

Prowadzący: dr inż. Robert Bembenik

Algorytm: CHAMELEON — Karypis G., Han E., Kumar V., *IEEE Computer* 32(8), 1999, str. 68–75

1. Wprowadzenie

1.1. Cel projektu

Celem jest opracowanie projektu implementacji oraz pełnej implementacji hierarchicznego algorytmu klasteryzacji **CHAMELEON** (Karypis, Han, Kumar 1999) w nowoczesnej konwencji Pythona, ze zgodnością ze scikit-learn API.

1.2. Miejsce CHAMELEON-a wśród algorytmów klasteryzacji

CHAMELEON jest hierarchicznym, aglomeracyjnym algorytmem klasteryzacji operującym na **grafie k-najbliższych sąsiadów**. W przeciwieństwie do wcześniejszych algorytmów hierarchicznych (CURE 1998, ROCK 1999, klasyczne single/complete/average linkage), które stosują **statyczny model klastra** (zbiór reprezentantów lub macierz podobieństw), CHAMELEON wprowadza **dynamiczne modelowanie** — kryterium łączenia dwóch klastrów uwzględnia ich **wewnętrzną charakterystykę** (gęstość, interconnectivity).

Dzięki temu algorytm poprawnie obsługuje klastry o **różnych kształtach, rozmiarach i gęstościach** — sytuacje, w których zawodzą k-means, DBSCAN, CURE i ROCK (dowodzone empirycznie w paperu Karypis 1999, sekcja 5, na zbiorach DS1–DS5).

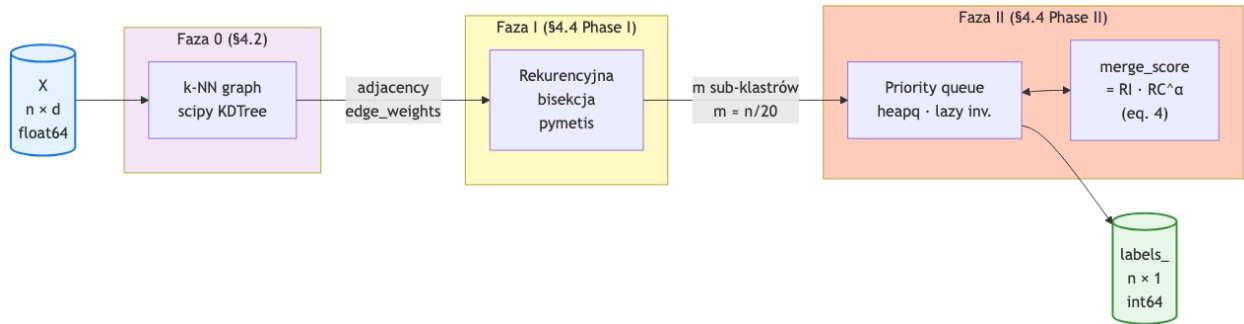
Kryterium decyzyjne fazy łączenia:

$$\text{score}(C_i, C_j) = RI(C_i, C_j) \cdot RC(C_i, C_j)^\alpha$$

gdzie RI (relative interconnectivity) i RC (relative closeness) są **znormalizowanymi** miarami w stosunku do wewnętrznej struktury klastrów (rozdz. 2).

2. Algorytm CHAMELEON

2.1. Schemat dwufazowy



Rysunek 1: Schemat przepływu danych w algorytmie CHAMELEON

Algorytm wykonuje 3 etapy: (0) budowa grafu k-NN; (I) rekurencyjna bisekcja grafu na m sub-klastrów; (II) aglomeracyjne łączenie sub-klastrów do osiągnięcia $n_clusters$.

2.2. Faza 0 — graf k-NN (§4.2)

Buduje rzadki graf $G = (V, E, w)$: krawędź (i, j) istnieje wtedy, gdy j jest jednym z k najbliższych sąsiadów i (lub odwrotnie). Waga krawędzi: $w(i, j) = 1 / d(x_i, x_j)$. Realizacja przez `scipy.spatial.KDTree.query` w czasie $O(n \log n)$ (vs naiwne $O(n^2)$ w implementacji referencyjnej).

2.3. Faza I — rekurencyjna bisekcja (§4.4 Phase I)

Dopóki istnieje sub-klaster o rozmiarze $> min_cluster_size$, dzielimy największy 2-way min-cutem. Realizacja przez `pymetis.part_graph(2, objtype='cut', ufactor=250)` (METIS multilevel, Karypis & Kumar 1998). Wynik: $m \approx n / 20$ sub-klastrów.

2.4. Faza II — aglomeracja (§4.4 Phase II)

Kolejka priorytetowa (heapq) z parami sąsiadujących sub-klastrów, kluczowanymi przez `merge_score`. Iteracyjnie pobieramy parę o najwyższym score, łączymy, aktualizujemy sąsiedztwo. **Lazy invalidation** (version counter na każdym klastrze) eliminuje potrzebę $O(m^2)$ rebuildu kolejki po każdym scaleniu.

2.5. Miary podobieństwa (§4.3)

Niech $EC_{\{C_i, C_j\}}$ — krawędzie krzyżujące granicę między klastrami; EC_{C_i} — bisektor klastra C_i (zbiór krawędzi przeciętych przy 2-way bisekcji C_i).

Relative Interconnectivity (eq. 1):

$$RI(C_i, C_j) = \frac{|EC_{C_i, C_j}|}{(|EC_{C_i}| + |EC_{C_j}|)/2}$$

Relative Closeness (eq. 2):

$$RC(C_i, C_j) = \frac{\bar{S}_{EC_{C_i, C_j}}}{\frac{|C_i|}{|C_i|+|C_j|}\bar{S}_{EC_{C_i}} + \frac{|C_j|}{|C_i|+|C_j|}\bar{S}_{EC_{C_j}}}$$

gdzie $\bar{S}_E$ — średnia waga krawędzi w zbiorze E.

Merge score (eq. 4):

$$\text{score}(C_i, C_j) = RI(C_i, C_j) \cdot RC(C_i, C_j)^\alpha$$

Wartość $\alpha > 1$ preferuje closeness; $\alpha < 1$ — interconnectivity. Domyślnie $\alpha = 2.0$.

2.6. Parametry

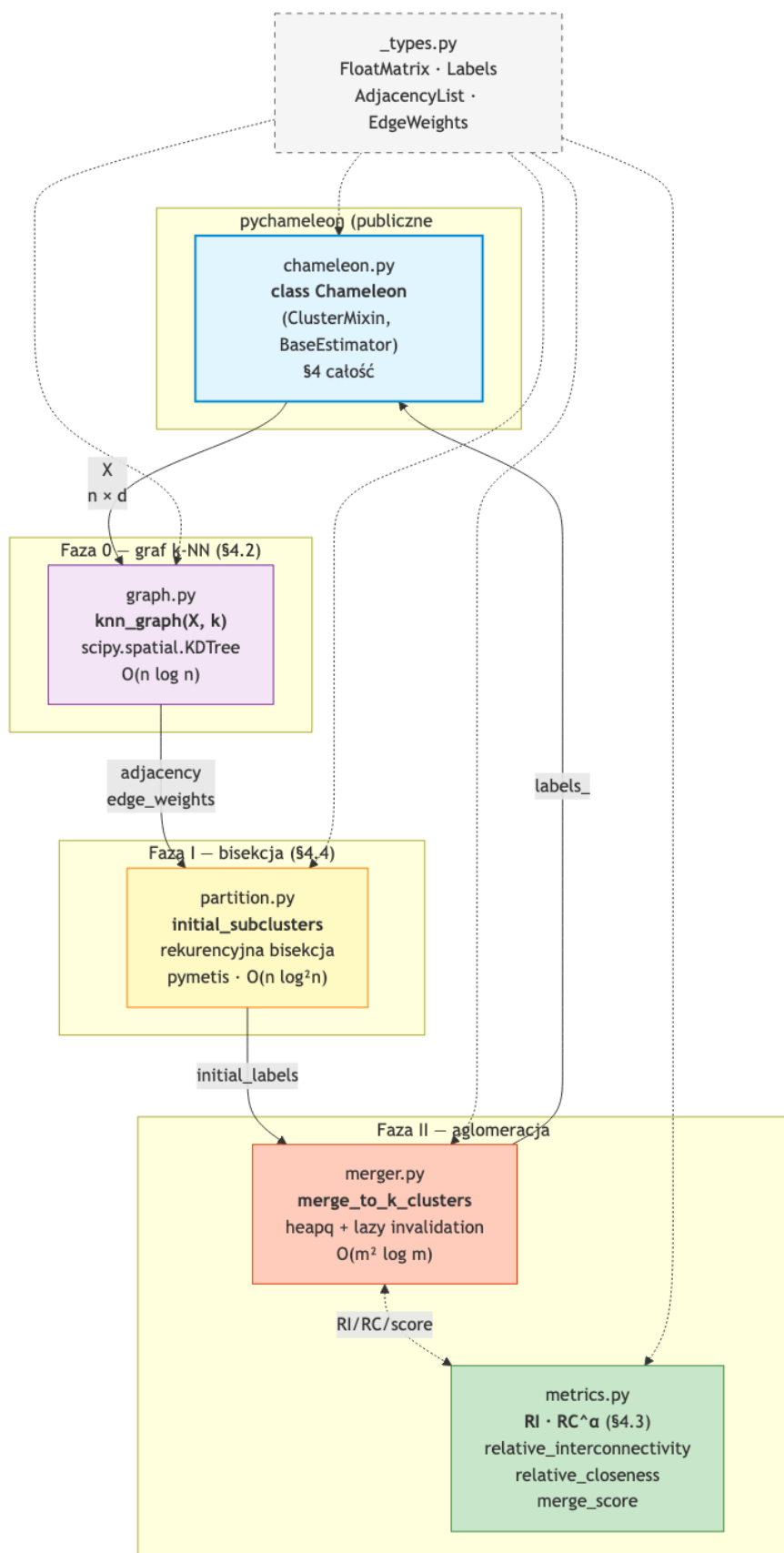
Parametr	Domyślnie	Wpływ
n_clusters	8	docelowa liczba klastrów
k_nn	10	gęstość grafu k-NN
min_cluster_size	0.025·n	minimalny rozmiar sub-klastra w fazie I
alpha	2.0	eksponent RC w merge_score

3. Struktura implementacji

3.1. Architektura — 5 modułów

Moduł	Sekcja paperu	Główne funkcje / klasy
graph.py	§4.2	knn_graph(X, k) → (adjacency, edge_weights)
partition.py	§4.4 Phase I	initial_subclusters(adj, weights, min_size) → labels
metrics.py	§4.3	relative_interconnectivity, relative_closeness, merge_score
merger.py	§4.4 Phase II	merge_to_k_clusters(adj, weights, init_labels, k, α) → labels
chameleon.py	§4 całość	class Chameleon(ClusterMixin, BaseEstimator)

_types.py dostarcza wspólnych aliasów (FloatMatrix, Labels, AdjacencyList, EdgeWeights). Publiczne API: jedna klasa Chameleon (from pychameleon import Chameleon).



Rysunek 2: Architektura pakietu pychameleon

3.2. Mapowanie kroków algorytmu na implementację

Krok algorytmu (paper)	Implementacja
§4.2 budowa grafu k-NN	KDTree.query(X, k+1), symetryzacja adjacency
§4.2 wagi krawędzi $1/d(x_i, x_j)$	weight = 1.0 / dist
§4.4 Phase I bisekcja największego sub-klastra	pymetis.part_graph(2, objtype='cut', ufactor=250)
§4.4 Phase II inicjalne wyliczenie score par	iteracja po parach sąsiadujących + push do heap
§4.4 Phase II wybór pary o max score	heapq.heappop z lazy invalidation (version cnt.)
§4.4 Phase II aktualizacja po scaleniu	push tylko sąsiadów scalonego super-klastra
§4.3 obliczenie EC_C_i (bisekcja wewnętrzna)	pymetis.part_graph(2, ...) z cache wyników

3.3. Decyzje projektowe

Decyzja	Wybór	Uzasadnienie
Build backend	hatchling	minimalny, nowoczesny pyproject.toml
Layout	src-layout	zapobiega bugom import-order (PEP 660)
Type checker	mypy --strict	standard sklearn-style libs
Lint	ruff	10x szybszy od pylint+flake8+isort razem
API wejścia	X (n × d, float64)	konwencja sklearn; pipeline-compatible
k-NN backend	scipy.spatial.KDTree	O(n log n), built-in dep sklearn
Graph repr.	list[ndarray] adj.	direct pymetis CSR; szybsze niż networkx

Wybór sklearn-compatible API od pierwszego commita motywowany długoterminowym celem publikacji jako pychameleon na PyPI (lato 2026); precedens: HDBSCAN przeniesiony z scikit-learn-contrib do sklearn core w 2023 r.

4. Struktury danych

4.1. Dane źródłowe (wejście)

Atrybut	Wartość
Klasa	numpy.ndarray
Kształt	(n_samples, n_features)
Dtype	np.float64
Walidacja	sklearn.utils._validate_data(accept_sparse=False)
Alias typu	FloatMatrix = NDArray[np.float64]

Walidacja delegowana do sklearn — automatyczna konwersja list/DataFrame, odrzucanie NaN/inf, ustawianie `n_features_in_` i `feature_names_in_`. Zbiory testowe wczytywane z plików CSV w `tests/data/` przez `np.loadtxt` (fixtures w `tests/conftest.py`).

4.2. Wewnętrzne struktury algorytmu

Struktura	Reprezentacja	Po co tak
Graf k-NN — sąsiedzi	<code>AdjacencyList = list[NDArray[int64]]</code>	direct pymetis CSR compatibility
Graf k-NN — wagi	<code>EdgeWeights = list[NDArray[float64]]</code>	równoległa indeksacja z adjacency
Etykiety klastrów	<code>Labels = NDArray[int64]</code> <code>shape(n,)</code>	maski boolean, sklearn convention
Kolejka priorytetowa (Faza II)	<code>heapq</code> z <code>(-score, ver_i, ver_j, ci, cj)</code>	$O(\log m)$ push/pop + lazy invalidation
Wersje klastrów	<code>dict[int, int]</code>	invalidacja przestarzałych wpisów heap'a
Cache <code> EC_Ci </code>	<code>dict[int, float]</code>	unikamy powtarzanych bisekcji pymetis

`adjacency[i]` to tablica indeksów sąsiadów wierzchołka `i`; `edge_weights[i][j]` to waga krawędzi `(i, adjacency[i][j])`. Reprezentacja symetryczna: jeśli `j ∈ adjacency[i]`, to również `i ∈ adjacency[j]` z tą samą wagą.

4.3. Format CSR dla pymetis

`pymetis.partition_graph(nparts, xadj, adjncy, eweights)` wymaga formatu Compressed Sparse Row z **całkowitymi** wagami. Konwersja `AdjacencyList` → CSR dokonywana w wywołaniach `partition.py` i `metrics.py`; wagi float skalowane przez `SCALE = 10000` (4 miejsca po przecinku).

4.4. Uzasadnienie wyborów

Adjacency list vs `scipy.sparse.csr_matrix`: wybrana adjacency list ze względu na **łatwiejszą iterację per-wierzchołek** (dominuje w Fazie II), a CSR i tak musimy generować dla pymetis.
heapq vs SortedList: `heapq` ze `stdlib` + lazy invalidation daje amortyzowany $O(\log m)$ bez zewnętrznej zależności.
numpy.ndarray vs list[int] dla labels: wektorowe maski (`labels == c`) są $O(n)$ zamiast $O(n)$ w pętli, plus konwencja sklearn.

5. Ocena efektywności

5.1. Złożoność teoretyczna (paper §4.5)

Niech `n` — liczba punktów, `m` — sub-klastrów po Fazie I (typowo $m \approx n/20$), `k` — parametr `k_nn`.

Faza	Nasza impl.	Moonpuck (referencyjna)
Faza 0 k-NN graph	$O(n \log n)$ KDTree	$O(n^2)$ naiwna macierz dist.
Faza I rekurencyjna bisekcja	$O(n \log^2 n)$	$O(n \log^2 n)$
Faza II aglomeracja	$O(m^2 \log m)$ lazy	$O(m^3)$ rebuild po scaleniu
Pamięć	$O(nd + nk + m^2)$	$O(n^2)$ (full distance mat.)

Sumarycznie: dominuje $O(n \log^2 n)$. Dla $n = 10^4$: ok. 1.7×10^5 operacji vs 10^8 przy $O(n^2)$ — różnica 3 rzędów wielkości.

5.2. Empiryczne benchmarki implementacji referencyjnej

Przygotowując projekt uruchomiłem implementację referencyjną Moonpuck ([link](#)) na 3 zbiorach. Wyniki w `benchmarks/reference_moonpuck/`:

Zbiór	n	k	k_nn	m	α	Runtime (s)	Bottleneck
aggregation	788	7	20	40	2.0	9.95	balanced
smileface	644	4	10	20	2.0	2.08	balanced
t4_8k	8000	6	20	40	2.0	674.47	naiwny $O(n^2)$ k-NN

Środowisko: MacBook Pro Apple Silicon, Python 3.11, `pymetis>=2023.1`.

5.3. Predykcja przyspieszenia

Główne planowane optymalizacje:

1. **scipy.spatial.KDTree** zamiast naiwnego k-NN — speedup $O(n^2) \rightarrow O(n \log n)$, asymptotycznie $\approx 47\times$ dla $n = 8000$.
2. **Cache_internal_interconnectivity** — eliminacja powtarzanych bisekcji; Moonpuck `recompute`uje przy każdym dostępie.
3. **Lazy invalidation kolejki** — zamiast $O(m^2)$ rebuildu po każdym scaleniu.
4. **numpy.ufuncs** zamiast pętli Pythona w `metrics.py` — stała $\sim 10\text{--}50\times$.

Spodziewany całkowity speedup vs Moonpuck: 50–100×

5.4. Ograniczenia

- **KDTree** efektywne tylko dla niskich wymiarów ($d \leq 10$); dla $d > 10$ koszt rośnie do $O(n)$ /zapytanie. Roadmap v0.3: zastąpienie przez Annoy/HNSW.
- **pymetis nie jest deterministyczny** (randomized refinement); `Chameleon().fit(X).labels_` może się minimalnie różnić między uruchomieniami. Mitigacja: w v0.2 parametr `random_state`.
- **Brak obsługi szumu** (zgodne z paperu Karypis 1999 — wszystkie punkty są przypisywane). Roadmap v0.2: parametr `min_samples_per_cluster` (analogia do HDBSCAN).

6. Plan testów

Strategia walidacji jest 5-warstwowa, realizowana w tests/ przy użyciu pytest, pytest-cov, numpy.testing, sklearn.utils.estimator_checks oraz sklearn.metrics.adjusted_rand_score.

Warstwa	Cel	Liczba / próg	Status (26.04)
Jednostkowe	Każdy moduł osobno; hand-computed wartości	~25 (5 modułów × ~5)	szkielet, treść w Etap 2
Integracyjne (e2e)	fit() na 3 datasetach	7 testów; ARI > 0.85 vs ground truth	12 testów skeleton pass
sklearn- compat	check_estimator(Chameleon())	~30 wewnętrznych	po implementacji fit()
Porównawcze	Wyniki vs Moonpuck na tych samyh parametrach	3 datasety × ARI > 0.9	Etap 2
Parametryczne	Sweep k_nn, α, min_cluster_size	3 sweep'y × ~5 wartości	Etap 2
Skalowalności	n vs runtime; slope log-log ≈ 1.1	6 punktów n ∈ [100, 50000]	Etap 2

Pokrycie kodu (target): `pytest --cov=pymetis` ≥ **90% linii**.

6.1. Testy jednostkowe — kluczowe asercje

- **test_graph.py:** symetria grafu ($j \in \text{adj}[i] \Leftrightarrow i \in \text{adj}[j]$), brak self-loops, $\text{weight} \approx 1/\text{distance}$, dla 3 odseparowanych blobów krawędzie tylko wewnątrz blobów.
- **test_partition.py:** wszystkie punkty otrzymują `label != -1`, każdy sub-klastr spełnia $|C| \geq \text{min_cluster_size}$, liczba sub-klastrów $m \approx n/\text{min_size}$.
- **test_metrics.py:** RI/RC/score liczone dla małych grafów (3–5 wierzchołków) porównywane z **ręcznie obliczonymi** wartościami referencyjnymi; `merge_score(α=0) == ri`.
- **test_merger.py:** zbieżność `len(unique(labels)) == target_k`; etykiety zwarte w `[0, k)`; lazy invalidation poprawnie pomija przestarzałe wpisy heap'a.

6.2. Tolerancja ARI > 0.9 w testach porównawczych

Próg 0.9 (a nie 1.0) ze względu na **niedeterministyczność pymetis** — randomized refinement w METIS Kernighan-Lin daje minimalnie różne bisekcje między uruchomieniami. Walidujemy zachowanie algorytmu, nie identyczność labels.

7. Zbiory danych testowych

Zbiór	n	dim	Klastrów	Ground truth	Charakterystyka
Aggregation	788	2	7	tak	klastry różnych rozmiarów, łączące się
smileface	644	2	4	tak	klastry niewypukłe (łuk uśmiechu)

Zbiór	n	dim	Klastrów	Ground truth	Charakterystyka
t4_8k	8000	2	6	wizualnie	klastry z dziurami i szumem tła (DS3 z paperu)
small_blobs	60	2	3	n/a (synth)	deterministyczna fixture, sanity check
make_blobs	var	2	5	tak	sklearn-generated, testy skalowalności

Zbiory referencyjne (Aggregation, smileface, t4_8k) są **vendored** w tests/data/. Pochodzenie: Aggregation — Gionis et al., *ACM TKDD* 2007; smileface — repozytorium Moonpuck; t4_8k — paper Karypis 1999 (zbiór DS3, fig. 4). small_blobs jest deterministyczną fixture w tests/conftest.py (3 centra, $\sigma=0.3$, seed=42). Dla testów skalowalności używamy sklearn.datasets.make_blobs(n_samples=n, centers=5, random_state=42) z $n \in \{100, 500, 1000, 5000, 10000, 50000\}$.

8. Bibliografia

- [1] **Karypis G., Han E.-H., Kumar V.** *CHAMELEON: A hierarchical clustering algorithm using dynamic modeling*. IEEE Computer 32(8), 1999, str. 68–75. [docs/chameleon_karypis_1999.pdf]
- [2] **Karypis G., Kumar V.** *A fast and high quality multilevel scheme for partitioning irregular graphs*. SIAM J. Sci. Comput. 20(1), 1998, str. 359–392. [METIS; docs/metis_karypis_1998.pdf]
- [3] **Bembenik R.** *Projekt MED 2026L: Wymagania i opis*. Politechnika Warszawska, 2026. [docs/Projekt_MED_2026L_rb.pdf]
- [4] **Moonpuck.** *chameleon_cluster — Python implementation of CHAMELEON*. Repozytorium GitHub, commit 1c0a65ee6a79706e4d415dd7ca78da5d3c29906d. https://github.com/Moonpuck/chameleon_cluster
- [5] **Gionis A., Mannila H., Tsaparas P.** *Clustering aggregation*. ACM TKDD 1(1), 2007. [pocho-dzenie zbioru Aggregation.csv]
- [6] **Pedregosa F. et al.** *Scikit-learn: Machine learning in Python*. JMLR 12, 2011, str. 2825–2830. [BaseEstimator / ClusterMixin / check_estimator]
- [7] **Virtanen P. et al.** *SciPy 1.0: Fundamental algorithms for scientific computing in Python*. Nature Methods 17, 2020, str. 261–272. [scipy.spatial.KDTree]
- [8] **Klockner A.** *PyMETIS: Python wrapper for METIS*. PyPI, <https://pypi.org/project/PyMetis/>.